

代数函数与图像语言实验

第007单元 先写对象再运行公式

这份实验把讲义中的每个运算顺序变成可核对的记录。你要先手算与画点，再让程序检查数字；遇到错误时说明究竟是数学上没有定义、单位用错、运算次序变了，还是计算机浮点范围不足。实验只用单个数、Python函数、列表与循环，先修001、003即可。不需要NumPy、矩阵或微积分。

完成后，你应得到一个函数值CSV和一份JSON结果，能解释两份数据里的空白与数字，也能用 $A=[1,9]$ 、 $B=[4,4]$ 说明三种目标选择的差异。所有数据是课程原创合成例子，不是模型评测结果。

一 准备与实际运行

进入本单元目录后运行下列命令。核心实验与测试只依赖Python3.12标准库，普通CPU即可。Notebook额外需要ipykernel与JupyterLab；environment.yml给出学习环境。绘图依赖是可选的，已生成的图可以直接阅读。

```
python experiment.py
python test_experiment.py
```

第一条命令读取data/input_grid.csv，写入outputs/function_values.csv、outputs/results.json和outputs/environment.json。第二条命令检查手算答案、故意失败的输入以及新进程重跑。程序使用脚本所在目录定位数据；从别的工作目录调用也应成功。成功测试会输出status为passed的JSON，而不仅是“没有红字”。

打开experiment.ipynb时，先重新启动内核，再执行所有单元格。按顺序执行是检查，不是可选的装饰：如果后面的结果依赖你先前随手定义的变量，别人就无法凭这份文件得到相同过程。发布包中的Notebook保留实际顺序执行输出；它不是空白模板。

建议先在纸上画两条坐标轴。横轴写输入及单位，纵轴写输出及单位。每完成一步，先记预测，再查看代码结果。你无需照着已有图片描线；你要说明为什么这些点应该处在那些位置。

二 三类方程不能只输出一个数字

从 $2x+1=7$ 开始，两边减1得到 $2x=6$ ，再除以非零数2得到 $x=3$ 。把3代回原式，左边确实为7。请在纸上把每一步允许的原因写在等号旁。

接着算 $0x+3=3$ 与 $0x+3=4$ 。前者左侧恒为3，在输入允许所有实数时，所有实数都满足；后者恒等式不成立，没有解。这两种情况都不能通过除以0来“求出x”。

```
from experiment import solve_affine
for a, b, c in [(2, 1, 7), (0, 3, 3), (0, 3, 4)]:
    print(solve_affine(a, b, c))
```

依次应得到kind为unique、all_real、no_solution。只有第一项value为3；另外两项value为None，表示这里不应装入一个唯一数值。若任务定义域另有约束，唯一代数解还要再检查是否在范围内。例如x限定为0到2时，方程 $2x+1=7$ 没有这个受限任务下的可行解；当前求解器默认实数定义域，不暗中替你添加设备限制。

三 先画点再谈定义域

手算x取0、1、2、3时的 $2x+1$ ，得到1、3、5、7。把(0,1)、(1,3)、(2,5)、(3,7)画在图上。沿横轴走1格，纵轴总增加2，这给出直线关系的数值解释。改变纸上横纵轴比例，线的视觉倾斜会改变，表里的变化率仍是每格2个读数单位。

现在把任务输入限制为0到5格。起点(0,1)与终点(5,11)都要画实心，因为边界包含在允许范围内。定义域是闭区间[0,5]，值域是[1,11]。函数表达式能计算100，并不证明装置能设到100格，也不证明真实校准关系在那里仍准确。

查看function_values.csv：输入网格有负数、0和正数。affine_2x_plus_1是 $2x+1$ ，square是 x^2 ，power_2是2的x次幂，rectifier是 $\max(0,x)$ 。log2在非正数对应行为空，log2_status标明outside_real_domain。这个空白是明确的数学边界记录，不等于数值0；真正的 $\log_2 1=0$ 另有defined状态。

四 同一长度换单位后保持预测

用1、2、3厘米分别代入 $y=2x+1$ ，输出为3、5、7。同一长度用毫米表示， $u=10x$ ，因此 $x=u/10$ 。替换整个x后得到 $y=0.2u+1$ 。你要同时换输入数字和系数，不能只改CSV列名。

```
from experiment import affine
for cm in [1, 2, 3]:
    mm = 10 * cm
    print(cm, mm, affine(cm), affine(mm, 0.2, 1), affine(mm))
```

每行第三列与第四列应一致；最后一列分别为21、41、61，暴露了把毫米值交给厘米系数的错误。在纸上写明系数单位：原系数是读数单位/厘米，新系数是读数单位/毫米。偏置1仍是读数单位，因为这里只更改输入单位。

如果还要对长度取对数，先写明参考长度。长度L除以1厘米，或除以10毫米，得到相同无量纲比值。若把分母随意换成1毫米，就改了参考尺度，得到的对数也会改变。

五 区分指数规则与对数规则

列出2的-2、-1、0、1、2、3次幂：1/4、1/2、1、2、4、8。每向右走一步都乘2，而不是加2。然后把相同对应关系反读： $\log_2(1/4)=-2$ ， $\log_2 1=0$ ， $\log_2 8=3$ 。对数为负不等于输入为负。

```
from experiment import power_two, real_log
for x in [-2, -1, 0, 1, 2, 3]:
    y = power_two(x)
    print(x, y, real_log(y))
print((-2) ** 2, -2 ** 2)
```

前三列依次是指数、结果、反读回的指数；最后应打印4与-4。不要把Python的^当成乘方。检验输入0、-1以及底数1、0、-2时，先预测为什么不允许，再观察ValueError。程序只处理实数对数，不能靠补0来使这些输入“成功”。

```
from experiment import real_log
for x, base in [(0, 2), (-1, 2), (2, 1), (2, 0)]:
    try:
        real_log(x, base)
    except ValueError as error:
        print(x, base, type(error).__name__)
```

底数1/2是合法的，与底数1不同。试算 $\log_{1/2} 2=-1$ 、 $\log_{1/2} 4=-2$ ；输入增大，对数反而减小。因此讨论排序时必须一起交代底数。

六 验证公式也要验证误用

将x=4、y=8代入乘积公式： $\log_2(4 \times 8)=5$ ， $\log_2 4 + \log_2 8=5$ 。它来自指数相乘时指数相加。再试“把乘积改成和”： $\log_2(4+4)=3$ ， $\log_2 4 + \log_2 4=4$ 。只需这个合法输入反例，就足以说明所谓加法公式并不普遍成立。

```
import math
from experiment import real_log
assert abs(real_log(4 * 8) - real_log(4) - real_log(8)) < 1e-12
assert real_log(4 + 4) != real_log(4) + real_log(4)
assert abs(real_log(4 ** 3) - 3 * real_log(4)) < 1e-12
assert abs(real_log(8, 10) - math.log10(8)) < 1e-12
print("product, counterexample, power and change-of-base checks passed")
```

最后一行核对换底得到的以10为底的对数与Python专用函数，采用绝对误差小于10的负12次方。这是本组小数值的检查容限，不是所有尺度都适用的精度保证。公式的普遍成立由讲义的符号推导解释，有限测试只能帮你发现实现与推导的不一致。

七 三种汇总顺序逐项比较

$A=[1,9]$ 与 $B=[4,4]$ 的数是两条正损失，统一约定越小越好。请分三列手算，再看程序：原均值、原均值后取 $\log_2$ 、每项先取 $\log_2$ 再平均。不要在同一列里混用这三种操作。

目标	A	B	较小的方案
算术平均	5	4	B
算术平均后取 $\log_2$	约2.322	2	B
每项 $\log_2$ 后平均	约1.585	2	A

```
from experiment import positive_loss_summary
for name, values in [("A", [1, 9]), ("B", [4, 4])]:
    print(name, positive_loss_summary(values))
```

第二列目标是在同一个正的整体上用递增函数变换，保留原顺序。第三列目标改了聚合方式；因为平均对数等于正几何平均的对数，A对应3而B对应4。此处没有宣布哪种损失定义“更正确”；要作选择，需要先说明任务究竟重视哪种误差汇总。

如果有人说“取对数不改变最优解”，请追问取的是哪一个正量、用的底数是多少、取完后是否仍按同一方向比较。这个实验的结论只覆盖已明确的两方案、小数值和运算规则。

八 求和与复合按计算顺序核对

令列表为 $[2,4,6]$ 。先给每个元素加1再求和，得到 $3+5+7=15$ ；先求和再加1，得到 $12+1=13$ 。数学编号 a_1 、 a_2 、 a_3 对应Python索引0、1、2，不能把符号下标直接抄成列表位置。

在 $f(x)=2x+1$ 、 $g(x)=x^2$ 下，从输入2出发： $f(g(2))$ 先平方成4，再得到9； $g(f(2))$ 先得到5，再平方成25。另一个复合 $\log_2(x-1)$ 要求内层 $x-1>0$ ，故 $x>1$ 。单独检查 $x>0$ 不够。

```
from experiment import affine, square, rectifier, real_log
values = [2, 4, 6]
print(sum(x + 1 for x in values), sum(values) + 1)
print(affine(square(2)), square(affine(2)))
assert [rectifier(x) for x in [-2, 0, 3]] == [0, 0, 3]
assert real_log(2 - 1) == 0
```

请在纸上画分段rectifier。负半轴贴着横轴，正半轴沿 $y=x$ ；在 $(0,0)$ 画实心点。它在0有定义且左右数值接上，不能因转折就说它没有函数值。求导问题留到后续单元。

九 故意失败与实验边界

最后比较两种失败。`real_log(-1)`违反实数定义域；`power_two(1024)`在数学上有正值，但本机浮点计算超出范围。`power_two(-1075)`若被底层浮点数舍入为0，也会被主动拒绝，避免把正指数函数错记为0。

```
from experiment import real_log, power_two
for function, value in [(real_log, -1), (power_two, 1024), (power_two, -1075)]:
    try:
        function(value)
    except (ValueError, ArithmeticError) as error:
        print(function.__name__, value, type(error).__name__)
```

程序还拒绝空损失列表、非正损失、NaN、无穷大、布尔值和字符串。它不是一个全面的数值分析软件：极端系数、非常接近1的底数、运算中的抵消以及测量不确定性，都需要额外处理。请不要把有限检查误写成“对所有实数输入都稳定”。

十 交付和自检

保留手绘图、实际运行的Notebook、两个数据输出文件和一段简短解释。解释至少包含三个不同原因的失败：单位系数不匹配、实数定义域错误、浮点范围不足。再用你自己的话说清A和B为什么在第三种目标下交换名次。

核对测试输出包含11组检查，其中有100组Fraction精确分数作为独立根值核验，也有正常值与错误输入测试。通过这些测试说明当前实现符合本实验约定；它不能从十个输入点证明整个数学定义域上的性质。图形只帮助观察，推导才说明合法的普遍关系。